

Code Explanation

Testing DLT Pipeline

This is a test suite for a data pipeline that simulates the functionality of Databricks Delta Live Tables (DLT). The test class `TestDLTPipeline` contains methods to test data transformations and aggregations.

Overview of the Code

The code is a unit test suite written in Python using the `unittest` framework. It tests a data pipeline that reads customer and order data, performs data transformations, and aggregates the results. The test suite creates a Spark session, generates sample data, and tests various data transformations and aggregations.

Step 1: Setting up the Test Environment

The `setUpClass` method is called before running any tests. It creates a Spark session with the necessary configurations for Delta Lake and creates temporary directories for test data.

```
@classmethod
def setUpClass(cls):
    """Create a Spark session for testing."""
    cls.spark = SparkSession.builder
        .appName("TestDLTPipeline")
        .master("local[*]")
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .getOrCreate()
```

Step 2: Creating Sample Data

The `setUpClass` method generates sample customer and order data, writes it to CSV files in temporary directories, and stores the paths to these directories.

```
# Create sample customer data
customer_data = [
    (1, "John Doe", "john@example.com", "North"),
    (2, "Jane Smith", "jane@example.com", "South")
]
customer_schema = ["CustId", "Name", "EmailId", "Region"]
customer_df = cls.spark.createDataFrame(customer_data, customer_schema)
customer_df.write.csv(cls.customer_data_path, header=True, mode="overwrite")

# Create sample order data
order_data = [
    (101, "Item1", 10.0, 2, "2023-01-01", 1),
    (102, "Item2", 15.0, 1, "2023-01-02", 2)
]
order_schema = ["OrderId", "ItemName", "PricePerUnit", "Qty", "Date", "CustId"]
```

```
order_df = cls.spark.createDataFrame(order_data, order_schema)
order_df.write.csv(cls.order_data_path, header=True, mode="overwrite")
```

Step 3: Testing Data Transformations

The `test\_data\_transformation` method reads the customer and order data, performs data transformations (adding a `TotalAmount` column and joining the data), and checks the results.

```
def test_data_transformation(self):
    """Test data transformations that would be done in DLT."""
    # Read customer data
    customer_df = self.spark.read.csv(self.customer_data_path, header=True, inferSchema=True)

    # Read order data
    order_df = self.spark.read.csv(self.order_data_path, header=True, inferSchema=True)

    # Add TotalAmount column to order data
    order_df = order_df.withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))

    # Join customer and order data
    joined_df = customer_df.join(order_df, "CustId", "inner")
```

Step 4: Verifying Join Results

The test checks that the join operation produced the expected results.

```
# Check that the join worked correctly
self.assertEqual(joined_df.count(), 2)

# Check that TotalAmount is calculated correctly
order1 = joined_df.filter(F.col("OrderId") == 101).first()
self.assertEqual(order1["TotalAmount"], 20.0) # 10.0 * 2

order2 = joined_df.filter(F.col("OrderId") == 102).first()
self.assertEqual(order2["TotalAmount"], 15.0) # 15.0 * 1
```

Step 5: Testing Aggregations

The test performs aggregations on the joined data and checks the results.

```
# Test aggregation for customer aggregate spend
agg_df = joined_df.groupBy("Name", "Date").agg(F.sum("TotalAmount").alias("TotalAmount"))

# Check aggregation results
self.assertEqual(agg_df.count(), 2)

john_spend = agg_df.filter(F.col("Name") == "John Doe").first()
self.assertEqual(john_spend["TotalAmount"], 20.0)
```

```
jane_spend = agg_df.filter(F.col("Name") == "Jane Smith").first()
self.assertEqual(jane_spend["TotalAmount"], 15.0)
```

Step 6: Cleaning up the Test Environment

The `tearDownClass` method is called after running all tests. It stops the Spark session and removes the temporary directories.

```
@classmethod
def tearDownClass(cls):
    """Clean up temporary directories and stop the Spark session."""
    shutil.rmtree(cls.temp_dir)
    cls.spark.stop()
```